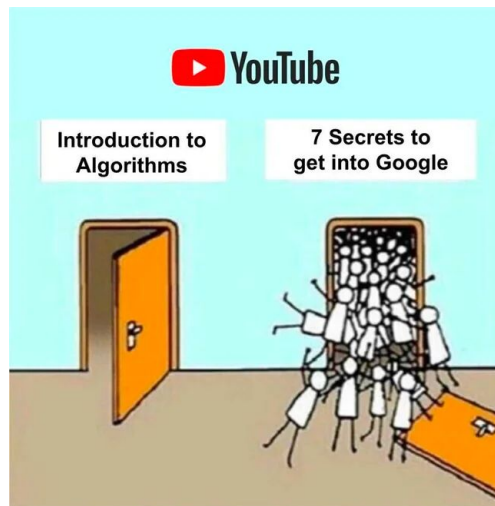


D2: Divide and Conquer and the Master Theorem



Sec 16: Fr 12:30pm-1:30pm FXB 1024
IA: Nick Frierler

Agenda

- ▶ Divide and conquer review
- ▶ Master theorem review
- ▶ Discussion problems

Divide and Conquer

Conquer but first divide

- ▶ (Not so) secretly a form of induction
- ▶ In reality, we should call it something like “Divide, Conquer, and Combine”
- ▶ **Divide**
 - ▶ Take your input and divide it into 2 or more subinputs
 - ▶ Repeat this until the problem becomes “trivial”
- ▶ **Conquer**
 - ▶ Now that all of our subsubsub...subinputs (base cases) are trivial to solve, CONQUER (SOLVE) THEM
- ▶ **Combine**
 - ▶ Take multiple solved subinputs and combine their solutions to get a solution for the input one recursive layer up

But how to prove my epic war strategy works?

- ▶ This is a proof by induction
- ▶ **Base Case**
 - ▶ Prove that the base case(s) are correct
 - ▶ Often very easy, so don't forget to do this step!
- ▶ **Inductive Step**
 - ▶ Suppose we split our input into k subinputs
 - ▶ Assume that the outputs of the recursive calls on those subinputs are all correct (**inductive hypothesis**), show that the step to **combine** those outputs into an output for the current input is correct
- ▶ Let induction work its magic

Ok my strategy works, but is it fast?

- ▶ We don't just want our algorithms to be correct
- ▶ We also want them to run *fast*
- ▶ **Runtime Analysis**
 - ▶ We need to derive a **recurrence relation** for the running time
 - ▶ Most of the divide and conquer algorithms we'll look at have recurrence relations that look like:
 - ▶ $T(n) = k * T(n/b) + O(n^d)$
 - ▶ We split an input of size n into k subinputs of size n/b and then combine the outputs of their recursive calls in at most $O(n^d)$ time (this also includes the time to divide inputs)
 - ▶ And then we use...

Master Theorem

Theorem's a pretty weird name for a Master...

- ▶ $T(n) = k * T(n/b) + O(n^d)$ where
 - ▶ $k \geq 1$
 - ▶ $b > 1$
 - ▶ $d \geq 0$
- ▶ If the $O(n^d)$ term is $\Theta(n^d)$ instead, we can replace the O s with Θ s

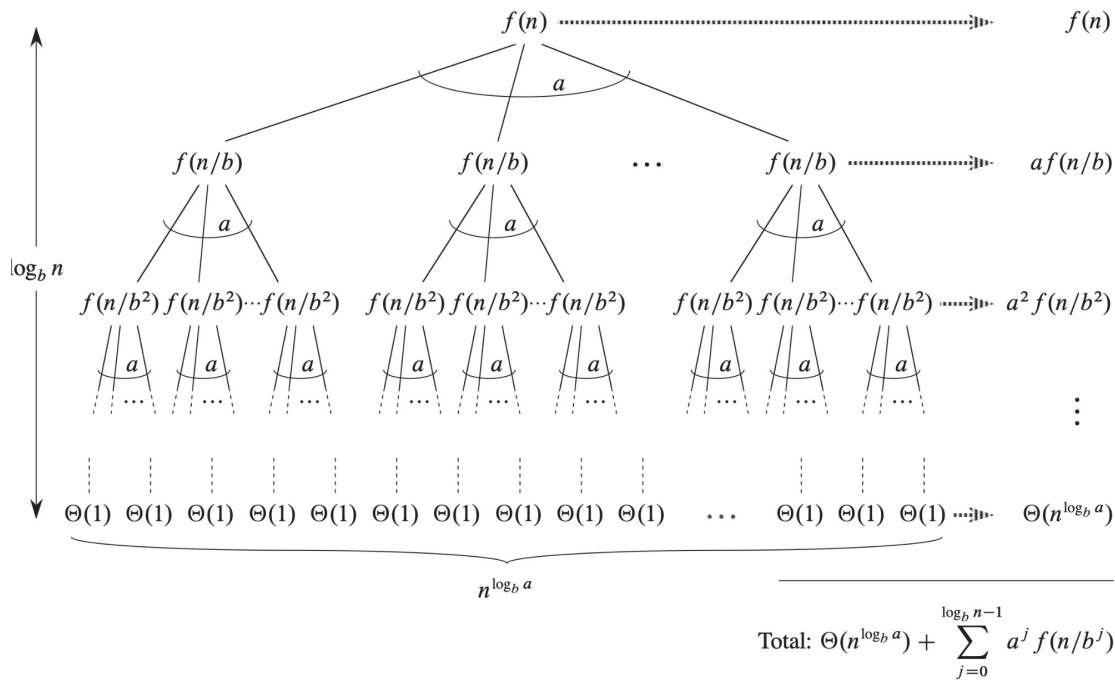
$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

Ok, but do you have any proof? Well...

4.6 Proof of the master theorem

99

- ▶ A proof of the Master Theorem isn't exactly difficult, but there's a lot to keep track of
 - ▶ It also isn't the most enlightening



Introduction to Algorithms, commonly called CLRS

But what about the real Master?

- ▶ $T(n) = k * T(n/b) + O(n^d * \log^w(n))$ where
 - ▶ $k \geq 1$
 - ▶ $b > 1$
 - ▶ $d \geq 0$
 - ▶ $w \geq 0$

$$T(n) = \begin{cases} \Theta(n^d \log^w n) & \text{if } k < b^d \\ \Theta(n^d \log^{w+1} n) & \text{if } k = b^d \\ \Theta(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

Any questions before
discussion problems?

Discussion Problems

Q1: MajorityElement

Given an array $A[1, \dots, n]$ of n elements, the **majority element** of A is the element that appears strictly more than $n/2$ times, if such an element exists. The algorithm **MajorityElement** on the next page outputs the majority element of A if it exists, otherwise it outputs a special symbol $\perp$. For simplicity we assume that n is a power of two, so that we can always split the array exactly in half.

Note that if a majority element exists, then it is unique—so we say “the majority element” instead of “a majority element”.

Q1: Majority Element

Give a recurrence and closed-form solution for the running time of **MajorityElement**

$$T(n) = k * T(n/b) + O(n^d)$$

$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

Q1: Majority Element

Give a recurrence and closed-form solution for the running time of **MajorityElement**

$$T(n) = k * T(n/b) + O(n^d)$$

$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

```

1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 

```

We do **2 recursive calls** of **size $n/2$** . We then iterate over A linearly, so we do **$O(n)$** work to combine our recursive outputs

$$T(n) = 2 * T(n/2) + O(n) \rightarrow T(n) = O(n * \log(n))$$

Q1: Majority Element

Show the correctness of

MajorityElement by first proving:

If z is the majority element of an array $A[1, \dots, n]$, then z must be the majority element of at least one of $A[1, \dots, n/2]$ and $A[n/2 + 1, \dots, n]$

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

Q1: Majority Element

Show the correctness of

MajorityElement by first proving:

If z is the majority element of an array $A[1, \dots, n]$, then z must be the majority element of at least one of $A[1, \dots, n/2]$ and $A[n/2 + 1, \dots, n]$

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

If z is the majority element of $A[1, \dots, n]$, then **there are at least $n/2 + 1$ occurrences of z** . At least $\text{ceil}((n/2 + 1)/2) = n/4 + 1$ of these must occur in one of $A[1, \dots, n/2]$ and $A[n/2 + 1, \dots, n]$. This is **more than half** of the elements in each of the subarrays. Therefore, z is a majority element of one of the 2 subarrays. Then, in **MajorityElement**, **one of x or y** must be the majority element of $A[1, \dots, n]$

Q2: Man this sort is slow

Consider the sorting algorithm slowsort, which can be represented with the following pseudocode. What is the most precise recurrence relation for the running time? Does the Master Theorem apply, and if so, what closed-form solution does it give?

```
1: function SLOWSORT( $A[1, 2, \dots, n]$ ) //  $n$  is length of  $A$ 
2:   SLOWSORT( $A[1, \dots, \lfloor \frac{n}{2} \rfloor]$ ) // sort both halves of the array recursively
3:   SLOWSORT( $A[\lfloor \frac{n}{2} \rfloor + 1, \dots, n]$ )
4:   if  $A[\lfloor \frac{n}{2} \rfloor] > A[n]$  then // largest item in first half is greater than largest in the second
5:     swap  $A[\lfloor \frac{n}{2} \rfloor]$  and  $A[n]$  // put largest item in the unsorted array at the end
6:   SLOWSORT( $A[1, \dots, n - 1]$ ) // recursively sort the entire array minus one element
7:   return
```

- ☐ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; the Master Theorem gives $T(n) = O(n^2 \log n)$
- ☐ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem gives $T(n) = O(n^{\log_2 3})$
- ☐ This cannot be written as a relation; the Master Theorem is unusable

Q2: Man this sort is slow

Consider the sorting algorithm slowsort, which can be represented with the following pseudocode. What is the most precise recurrence relation for the running time? Does the Master Theorem apply, and if so, what closed-form solution does it give?

```

1: function SLOWSORT( $A[1, 2, \dots, n]$ ) //  $n$  is length of  $A$ 
2:   SLOWSORT( $A[1, \dots, \lfloor \frac{n}{2} \rfloor]$ ) // sort both halves of the array recursively
3:   SLOWSORT( $A[\lfloor \frac{n}{2} \rfloor + 1, \dots, n]$ )
4:   if  $A[\lfloor \frac{n}{2} \rfloor] > A[n]$  then // largest item in first half is greater than largest in the second
5:     swap  $A[\lfloor \frac{n}{2} \rfloor]$  and  $A[n]$  // put largest item in the unsorted array at the end
6:   SLOWSORT( $A[1, \dots, n - 1]$ ) // recursively sort the entire array minus one element
7:   return
  
```

- ☐ $T(n) = 2T(\frac{n}{2}) + T(n - 1) + O(1)$; the Master Theorem gives $T(n) = O(n^2 \log n)$
- ☒ $T(n) = 2T(\frac{n}{2}) + T(n - 1) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T(\frac{n}{2}) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T(\frac{n}{2}) + O(1)$; the Master Theorem gives $T(n) = O(n^{\log_2 3})$
- ☐ This cannot be written as a relation; the Master Theorem is unusable

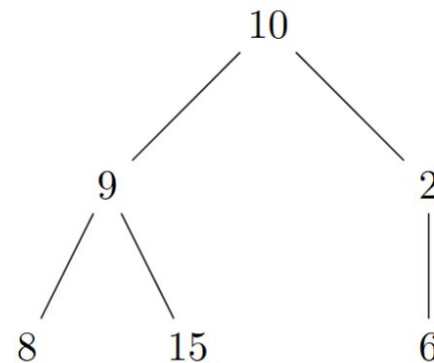
SlowSort does 3 recursive calls, 2 of size $n/2$, 1 of size $n-1$. We can see that only A and B are possibilities. The Master Theorem **does not work** with recursions that have terms that look like $T(n - k)$ for constants k . Therefore, the answer is **B**

Attendance



Q3: Complete binary tree local max

A **complete binary tree** is a binary tree in which every level, except possibly the last (leaf) level, is completely filled, and all the nodes in the last level are as far left as possible. Consider a complete binary tree $T = (V, E, r)$ rooted at r where each vertex is labelled with an integer. A vertex $v \in V$ is called a **local maximum** if the label of v is greater than or equal to the labels of all its adjacent nodes (parent and children nodes)



In this example, the local maxima are 10, 15, and 6.

Suppose you are given such a tree where the labelling is given implicitly (*you need to visit the vertex and query it to get its label*). Give an algorithm that computes a local maximum of T using just $O(\log |V|)$ vertex-label queries.

Note: T may contain multiple local maxima; the algorithm just needs to return one of them.

Q3: Complete binary tree local max

Input: complete, rooted, vertex-labeled binary tree $T = (V, E, r)$

Output: local max vertex

LocalMax($T = (V, E, r)$)

- ▶ Query r and its (up to) 2 children
- ▶ If the label of r is greater than or equal to both its children
 - ▶ Return r
- ▶ Else
 - ▶ Let u, v be r 's children and let u be a child with label at least as large as r
 - ▶ Let T' be the complete, vertex-labeled binary tree rooted at u
 - ▶ Return LocalMax(T')

▶ Correctness

- ▶ Note that every such binary tree has a local max
- ▶ Therefore, T' has a local max
- ▶ Since u 's label $\geq r$'s label, the local max of T' is also a local max of T

▶ Number of queries

- ▶ We do (at most) 3 queries per level
- ▶ There are $O(\log |V|)$ levels

Q4: Multiplication? Isn't that what calculators are for?

In lecture, you learned that splitting up big numbers into smaller chunks can make multiplication more efficient. In particular, you analyzed Karatsuba's algorithm which splits integers of length n into two integers of length $n/2$ and does 3 multiplications (whereas a naïve algorithm would use 4 multiplications). Recall that the running time of Karatsuba's algorithm is $O(n^{\log_2(3)})$ where $\log_2(3) \approx 1.585$.

Let algorithm A split integers of length n into 3 integers of length $n/3$ and do 9 multiplications.

Running time?

Let algorithm B split integers of length n into 3 integers of length $n/3$ and do 5 multiplications.

Running time?

Q4: Multiplication? Isn't that what calculators are for?

In lecture, you learned that splitting up big numbers into smaller chunks can make multiplication more efficient. In particular, you analyzed Karatsuba's algorithm which splits integers of length n into two integers of length $n/2$ and does 3 multiplications (whereas a naïve algorithm would use 4 multiplications). Recall that the running time of Karatsuba's algorithm is $O(n^{\log_2(3)})$ where $\log_2(3) \approx 1.585$.

Let algorithm A split integers of length n into 3 integers of length $n/3$ and do 9 multiplications.

Running time?

$$T(n) = 9 * T(n/3) + O(n) \rightarrow T(n) = O(n^2)$$

Let algorithm B split integers of length n into 3 integers of length $n/3$ and do 5 multiplications.

Running time?

$$T(n) = 5 * T(n/3) + O(n) \rightarrow T(n) = O(n^{\log_3(5)}) = O(n^{1.465})$$